

Aula 2

UML - Casos de uso, classes, sequencia, atividades, componentes e implantacao

Material autoral orientado ao edital TJSC 2026 - banca FGV

Foco da aula	Converter sua vivencia pratica em Java/Quarkus, Angular/TypeScript, APIs e sistemas corporativos em acerto de prova sobre modelagem UML.
ROI para prova	Alto: UML aparece expressamente no edital e costuma ser cobrada pela FGV por associacao entre diagrama, finalidade, elementos e semantica de relacionamento.
Modo de estudo	Estude primeiro o quadro comparativo, depois resolva as 20 questoes sem olhar o gabarito. Use os comentarios para montar caderno de erros.

1. Roteiro da aula e encaixe no edital

Esta aula cobre a parte de UML - Unified Modeling Language que aparece no bloco de Engenharia de Software e Desenvolvimento do edital: casos de uso, classes, sequencia, atividades, componentes e implantacao. Embora o edital mencione UML versao 2.1, a banca costuma usar linguagem mais moderna de UML 2.x em provas recentes. Para prova, o nucleo cobrado e estavel: identificar a finalidade do diagrama, interpretar relacionamentos e evitar confundir visao estatica, dinamica, funcional e fisica.

A FGV tem preferencia por enunciados contextualizados: um analista, um orgao publico, um sistema corporativo, um diagrama descrito ou apresentado, e alternativas que misturam termos muito proximos. Por isso, as questoes desta aula foram feitas com enunciados mais longos, casos institucionais e distratores fortes, sem copiar questoes reais.

Tópico	O que voce precisa dominar	Como a FGV costuma confundir
Caso de uso	Atores externos, fronteira do sistema, funcionalidades sob o ponto de vista do usuario e relacoes include/extend/generalizacao.	Trocar caso de uso por atividade, colocar ator dentro do sistema ou afirmar que caso de uso modela classe interna.
Classes	Estrutura estatica: classes, atributos, operacoes, visibilidade, associacao, multiplicidade, agregacao, composicao, generalizacao, realizacao e dependencia.	Confundir agregacao com composicao; confundir classe com objeto; ignorar multiplicidade.
Sequencia	Interacao entre objetos/participantes ao longo do tempo: lifelines, mensagens, ativacoes e fragmentos combinados.	Trocar com comunicacao, atividade ou estado; esquecer que a ordem temporal vertical e central.
Atividades	Fluxo de trabalho, decisoes, paralelismo, fork/join, merge, particoes e fluxo de objetos.	Confundir com fluxograma generico ou com diagrama de estados.
Componentes	Organizacao modular/logica de componentes, interfaces providas/requeridas, portas e dependencias.	Trocar componente por no fisico de implantacao.
Implantacao	Nos, dispositivos, ambientes de execucao, artefatos implantados e caminhos de comunicacao.	Trocar distribuicao fisica por estrutura logica ou por classes.

Como estudar esta aula em blocos curtos

- 1 Bloco 1 - 25 minutos: leia o quadro comparativo e os diagramas mentais. Marque as palavras-chave: usuario, estrutura estatica, tempo, fluxo, componente, no fisico.
- 2 Bloco 2 - 35 minutos: estude casos de uso, classes e sequencia. Esses tres concentram maior chance de pegadinha em prova.
- 3 Bloco 3 - 25 minutos: estude atividades, componentes e implantacao, sempre contrastando componente vs no, fluxo vs interacao, classe vs objeto.
- 4 Bloco 4 - 45 a 60 minutos: resolva as 20 questoes sem gabarito. Depois corrija e escreva no caderno de erros apenas a regra que faltou.

2. UML em uma frase: para que serve

UML - Unified Modeling Language - **e uma linguagem grafica de modelagem usada para visualizar, especificar, construir e documentar artefatos de sistemas de software.** A palavra decisiva para a prova e modelagem: **UML nao e metodologia**, nao e processo de desenvolvimento, nao e ferramenta CASE e nao e linguagem de programacao. Ela pode ser usada em cascata, RUP, Scrum, XP, projetos corporativos e documentacao arquitetural.

HACK DE PROVA: quando a alternativa disser que UML 'prescreve o ciclo de vida', 'executa código', 'substitui requisitos' ou 'define obrigatoriamente metodologia ágil', desconfie. UML modela; quem define processo e metodologia, não a UML.

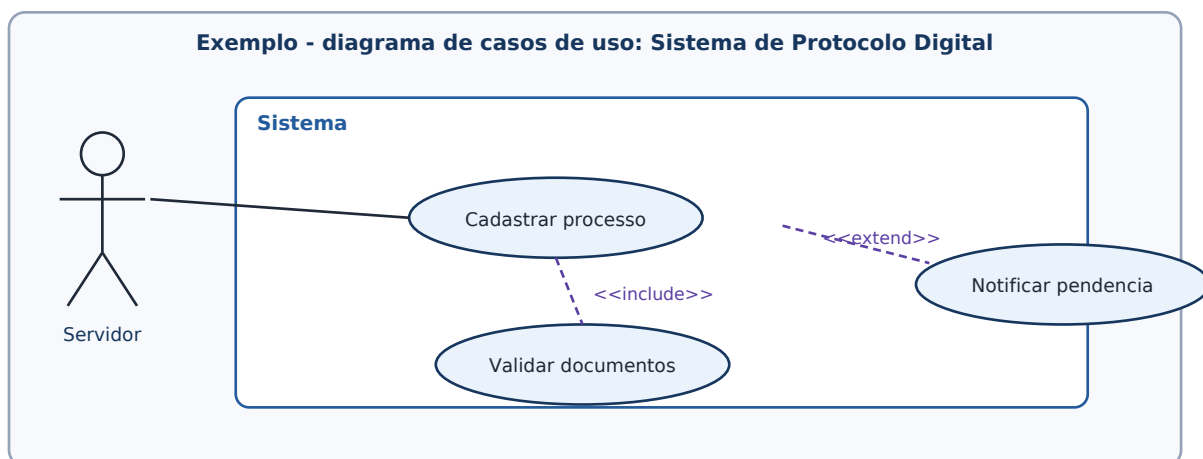
Classificação essencial: diagramas estruturais e comportamentais

A banca pode cobrar a família do diagrama. Em termos de memorização para prova, se o diagrama mostra **coisas e relações estáticas**, pense em **estrutural**; se mostra **comportamento, interação, fluxo ou tempo**, pense em **comportamental**. Sequência é um diagrama de interação, dentro da família comportamental.

Família	Diagramas desta aula	Pergunta que respondem	Palavra-chave
Estruturais	Classes, componentes e implantação	Do que o sistema é composto? Como seus elementos se relacionam? Onde os artefatos rodam?	estrutura
Comportamentais	Casos de uso e atividades	Quais funcionalidades existem? Qual o fluxo de atividades?	comportamento
Interação	Sequência	Quem fala com quem e em que ordem no tempo?	mensagens

3. Diagrama de casos de uso

O diagrama de casos de uso **representa funcionalidades do sistema sob o ponto de vista de atores externos**. Ele é útil em requisitos porque **ajuda a enxergar objetivos do usuário, fronteira do sistema e serviços que o sistema deve oferecer**. Não detalha algoritmo interno, não modela banco de dados e não descreve sequência temporal fina das chamadas.



Elementos que a FGV gosta de cobrar

Elemento	Significado de prova	Pegadinha
Ator	Papel externo que interage com o sistema: pessoa, outro sistema, serviço ou organização.	Ator não fica dentro da fronteira do sistema; não é necessariamente humano.
Caso de uso	Funcionalidade ou objetivo observável oferecido pelo sistema ao ator.	Não é classe, tela ou método Java; é comportamento funcional.
Fronteira do sistema	Limite do que pertence ao sistema modelado.	Atores ficam fora; casos de uso ficam dentro.
Associação	Comunica ator e caso de uso.	Não é herança nem dependência técnica de classe.
<<include>>	Um caso de uso base sempre incorpora comportamento comum de outro caso de uso.	Use quando o comportamento incluído é obrigatório/reutilizado.
<<extend>>	Um caso de uso adiciona comportamento opcional/condicional a outro.	A seta vai do caso extensor para o caso base; é condicional.
Generalização	Especialização entre atores ou entre casos de uso.	Não confundir com include/extend.

Conexão com sua prática

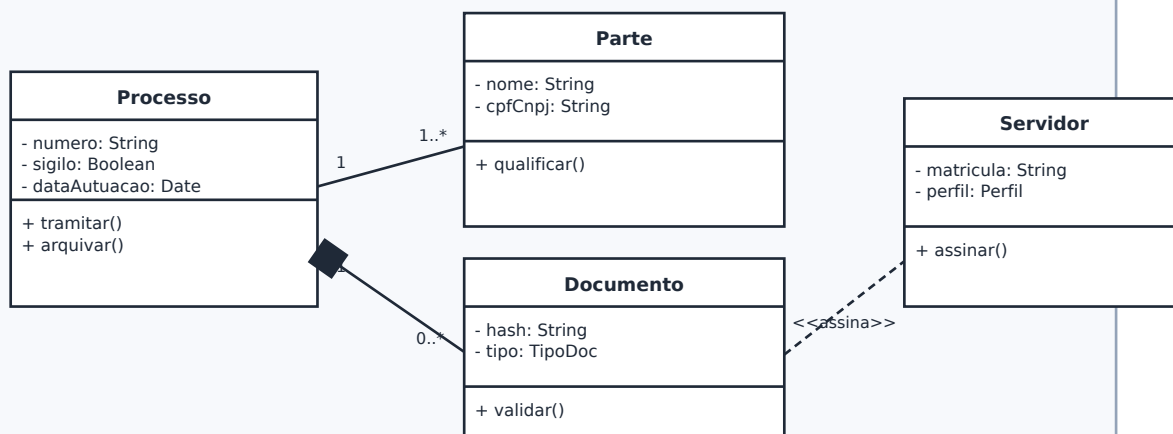
Em um sistema Angular + Quarkus, um caso de uso não é o componente Angular ConsultaProcessoComponent nem o endpoint GET /processos/{id}. Ele poderia ser 'Consultar processo', 'Protocolar petição' ou 'Assinar documento'. O endpoint, a tela e a classe de serviço são decisões de projeto/implementação; o caso de uso está mais próximo do objetivo funcional do ator.

Regra que vale ponto: caso de uso = funcionalidade visível para ator externo. Se aparecer 'ponto de vista do usuário', 'agente externo' ou 'interação entre usuário e sistema', a resposta tende a ser diagrama de casos de uso.

4. Diagrama de classes

O diagrama de classes é provavelmente o mais cobrado de UML em provas de desenvolvimento. Ele fornece uma **visão estática da estrutura do sistema**: classes, atributos, operações, associações, multiplicidades e relações como herança, composição, agregação, dependência e realização. Para o candidato desenvolvedor, a armadilha é trazer intuição de código e esquecer a semântica gráfica da UML.

Exemplo - diagrama de classes: domínio de tramitação



Classe, objeto e atributos

Uma classe é uma abstração ou tipo. Um objeto é uma instância concreta dessa classe em determinado momento. Em Java, a classe `Processo` define campos e métodos; um processo específico número 0001234-56.2026.8.24.0000 seria uma instância. A FGV adora perguntar: **se quer mostrar instâncias em um momento, o diagrama mais adequado é o de objetos**, não o de classes. Mas nesta aula o foco é classe.

Notação	Leitura	Exemplo
+	público	+ consultar(): ProcessoDTO
-	privado	- senha: String
#	protegido	# validarInternamente()
~	pacote	~ recalcularIndice()
atributo: Tipo	nome e tipo do atributo	dataAutuacao: LocalDate
operacao(param: Tipo): Retorno	assinatura de método/operacao	assinar(doc: Documento): Assinatura

Relacionamentos em classe: onde mora a maioria das pegadinhas

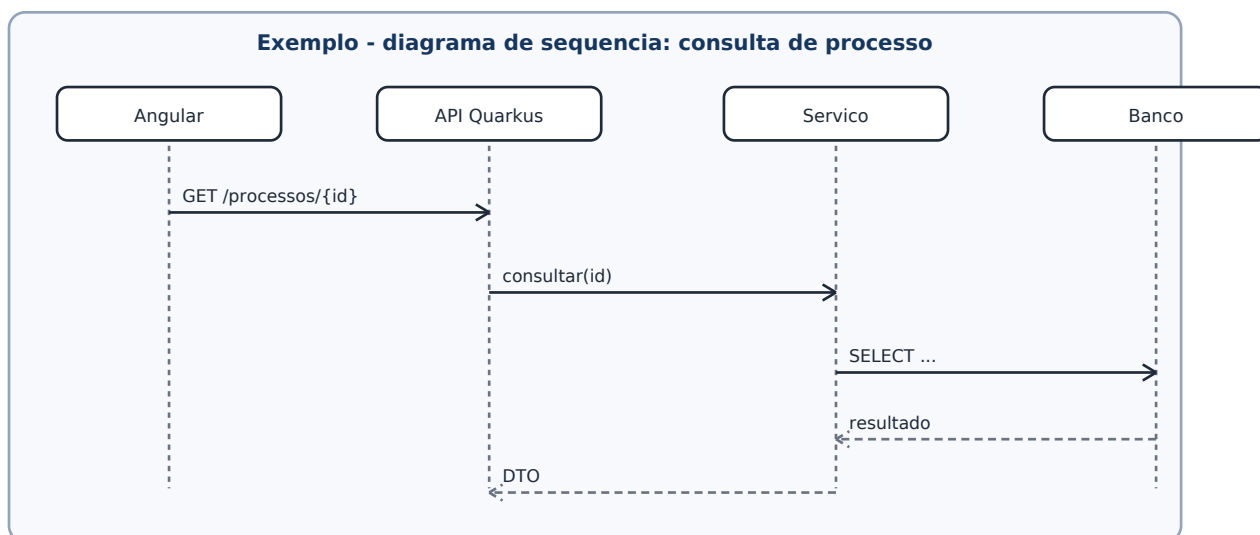
Relacionamento	Notacao mental	Significado	Pegadinha FGV
Associacao	linha simples	Ligacao estrutural entre classes; pode ter multiplicidade e navegabilidade.	Nao implica posse forte nem ciclo de vida.
Agregacao	diamante vazio	Relacao todo-parte fraca; a parte pode existir separadamente.	Nao confundir com composicao.
Composicao	diamante preenchido	Todo-parte forte; parte pertence ao todo e seu ciclo de vida costuma depender do todo.	Diamante fica no lado do todo.
Generalizacao	triangulo vazio apontando para superclasse	Heranca/especializacao: 'e um tipo de'.	Seta aponta para o mais geral.
Realizacao	linha tracejada + triangulo vazio	Classe/componente implementa interface/contrato.	Muito comum em interface.
Dependencia	linha tracejada com seta	Uso temporario; mudanca em um elemento pode afetar outro.	Mais fraca que associacao.
Classe associativa	classe ligada a uma associacao	Quando a propria relacao tem atributos.	Ex.: Alocacao tem gratificacao entre Servidor e Unidade.

Multiplicidade: decore como restricao de quantidade

Multiplicidade	Leitura	Exemplo de prova
1	exatamente um	Todo processo possui exatamente uma classe processual principal.
0..1	zero ou um	Um processo pode ou nao possuir segredo de justica.
* ou 0..*	zero ou muitos	Uma parte pode ter muitos processos.
1..*	um ou muitos	Um processo deve ter pelo menos uma parte.
m..n	intervalo minimo-maximo	Um colegiado pode ter de 3 a 7 membros.

5. Diagrama de sequencia

O diagrama de sequencia mostra como participantes interagem por mensagens ao longo do tempo. A leitura padrao e de **cima para baixo**. A **horizontal separa participantes/lifelines**; a **vertical mostra evolucao temporal**. Para sistemas corporativos, pense no fluxo Angular -> API Quarkus -> servico de dominio -> repositorio -> banco, mas lembre que o diagrama representa interacao, nao estrutura estatica de classes.



Elementos de prova

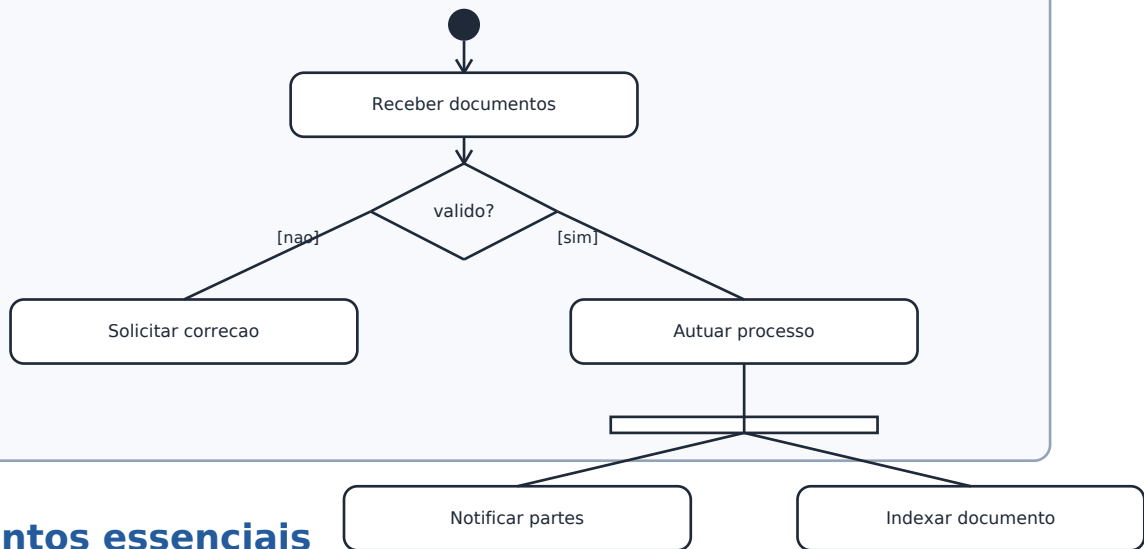
Elemento	O que significa	Como cair
Lifeline	Participante/objeto que existe durante a interacao.	Retangulos no topo com linha vertical tracejada.
Mensagem sincrona	Chamador espera retorno; comum em chamada de metodo/HTTP bloqueante.	Seta com cabeca preenchida em algumas notacoes.
Mensagem assincrona	Chamador nao espera necessariamente resposta imediata; comum em eventos/mensageria.	FGV pode associar a fila, evento, callback.
Execucao/ativacao	Periodo em que participante executa operacao.	Retangulo fino sobre a lifeline.
Retorno	Resposta da chamada; geralmente linha tracejada.	Nao confundir com nova chamada.
Fragmento alt	Alternativas condicionais, como if/else.	Guards [condicao] separam possibilidades.
Fragmento opt	Comportamento opcional, como if sem else.	Executa apenas se a guarda for verdadeira.
Fragmento loop	Repeticao.	Equivale a iteracao enquanto condicao for satisfeita.

HACK DE PROVA: se o enunciado falar em 'objetos interagem entre si ao longo do tempo', a resposta natural e diagrama de sequencia. Se falar em 'estrutura estatica', e classes. Se falar em 'fluxo de atividades', e atividade.

6. Diagrama de atividades

O diagrama de atividades **modela o fluxo de trabalho ou fluxo de controle/objetos entre atividades**. Ele e muito usado para processos de negocio, **logica de alto nivel**, **workflows** e **procedimentos com decisoes**, **paralelismo** e **sincronizacao**. Para prova, associe com workflow, processo, tarefas, fluxo de controle, decisoes e concorrencia.

Exemplo - diagrama de atividades: protocolo de peticao



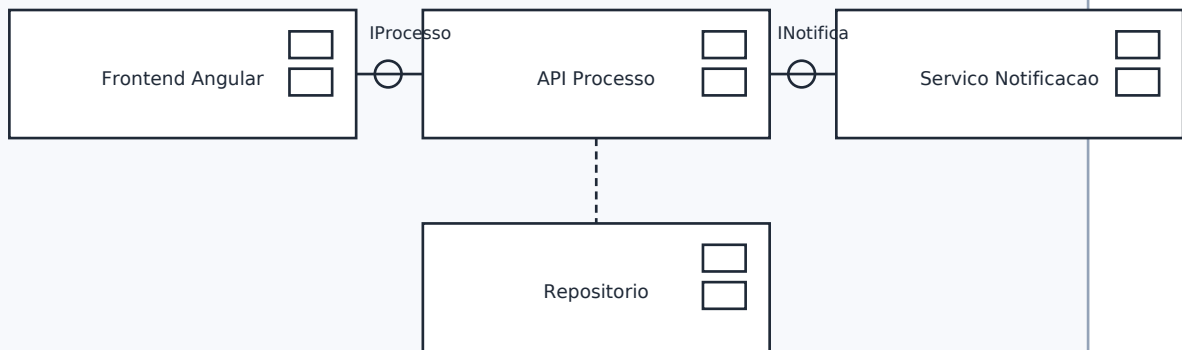
Elementos essenciais

Elemento	Leitura	Pegadinha
No inicial	Inicio do fluxo; circulo preenchido.	Nao e estado inicial de maquina de estados, embora a notacao seja parecida.
No final	Fim da atividade ou fim de fluxo.	Pode haver final de atividade e final de fluxo.
Acao/atividade	Unidade executavel do fluxo.	Nao e classe nem metodo necessariamente.
Fluxo de controle	Ordem de execucao das atividades.	Nao mostra mensagens temporais como sequencia.
Decisao	Escolha entre caminhos; losango com guardas.	Decision separa; merge junta alternativas.
Fork	Divide fluxo em ramos paralelos.	Barra grossa: um entra, varios saem.
Join	Sincroniza ramos paralelos.	Barra grossa: varios entram, um sai.
Particoes/swimlanes	Responsaveis por atividades: setor, ator, sistema.	Nao significam necessariamente classes.

7. Diagrama de componentes

O diagrama de componentes representa a organizacao modular do sistema: componentes, interfaces providas/requeridas, portas e dependencias. Ele fica entre a visao de projeto e a arquitetura logica. Em uma solucao corporativa, pode mostrar o componente 'Frontend Angular', 'API de Processos', 'Servico de Notificacoes', 'Modulo de Assinatura' e suas interfaces.

Exemplo - diagrama de componentes: arquitetura logica



Termo	Tradução de prova	Exemplo prático
Componente	Unidade modular substituível/implantável/lógica que encapsula comportamento e oferece interfaces.	API de petição, módulo de autenticação, serviço de notificação.
Interface provida	Serviço que o componente oferece para outros.	IConsultaProcesso exposta pela API.
Interface requerida	Serviço de que o componente precisa para funcionar.	Componente de processo requer INotificação.
Porta	Ponto de interação específico do componente.	Porta REST, porta de mensageria, porta de integração.
Dependência	Um componente depende de outro contrato/serviço.	Frontend depende da API, API depende do repositório.

8. Diagrama de implantação

O diagrama de implantação mostra a distribuição física dos artefatos de software em nós de hardware ou ambientes de execução. Aqui aparecem dispositivos, servidores, containers, máquinas virtuais, pods, ambientes de execução, artefatos implantados e caminhos de comunicação. O objetivo é responder: onde cada parte roda e como se comunica?

Exemplo - diagrama de implantação: distribuição física



Elemento	Significado	Exemplo
Node - no	Recurso computacional fisico ou ambiente de execucao.	Servidor Linux, cluster Kubernetes, pod, JVM, browser.
Device	No fisico de hardware.	Notebook, servidor, appliance.
Execution environment	Ambiente que executa artefatos.	JVM - Java Virtual Machine, container, application server.
Artifact	Arquivo/artefato implantado em um no.	api.jar, app.js, imagem Docker, schema.sql.
Communication path	Caminho de comunicacao entre nos.	HTTPS, JDBC, AMQP, REST.

Pegadinha central: componente e logico/modular; implantacao e fisico/runtime. Se o enunciado fala em servidor, dispositivo, ambiente de execucao, artefato implantado, pod, JVM, maquina virtual ou comunicacao entre nos, pense em implantacao.

9. Comparativo de prova: o que responder quando aparecer a palavra-chave

Palavra no enunciado	Resposta provavel	Motivo
ponto de vista do usuario	caso de uso	Funcionalidades e interacoes ator-sistema.
atores externos	caso de uso	Atores representam papeis externos.
classes, atributos e metodos	classes	Estrutura estatica do modelo.
multiplicidade, agregacao, composicao	classes	Relacionamentos estruturais.
objetos interagem ao longo do tempo	sequencia	Mensagens e ordem temporal.
workflow, fluxo de tarefas, processo	atividades	Fluxo de controle/objetos.
modulos e interfaces	componentes	Componentes e contratos logicos.
hardware, servidor, JVM, artefato implantado	implantacao	Distribuicao fisica/runtime.

Mapa mental textual

UML para prova FGV = para que serve o diagrama? Use a arvore: usuario externo -> caso de uso; classe/atributo/relacao -> classes; tempo/mensagem -> sequencia; workflow/decisao/fork -> atividade; modulo/interface -> componente; servidor/ambiente/artefato implantado -> implantacao.

Siglas e termos técnicos da aula

Sigla/termo	Expansão	Memorização
UML	Unified Modeling Language	Linguagem unificada de modelagem; não é metodologia.
OMG	Object Management Group	Consortio responsável por especificações como UML.
CASE	Computer-Aided Software Engineering	Ferramentas que apoiam engenharia de software e modelagem.
API	Application Programming Interface	Interface de programação; pode aparecer em componente ou sequência.
JVM	Java Virtual Machine	Ambiente de execução: cai em implantação.
REST	Representational State Transfer	Estilo arquitetural de APIs; pode aparecer em sequência/componente/implantação.
DTO	Data Transfer Object	Objeto de transporte; pode aparecer em sequência como retorno.
CRUD	Create, Read, Update, Delete	Operações funcionais; não confundir com casos de uso automaticamente.

10. Pegadinhas FGV - leitura rápida antes das questões

- UML não é metodologia, processo, IDE, linguagem de programação ou ferramenta CASE.
- Ator em caso de uso fica fora da fronteira do sistema e representa papel, não pessoa específica.
- Caso de uso representa funcionalidade percebida pelo ator, não tela, método, classe ou tabela.
- <<include>> tende a comportamento obrigatório/reutilizável; <<extend>> tende a comportamento opcional/condicional.
- Diagrama de classes é estrutural/estático; diagrama de sequência é temporal/dinâmico/interacional.
- Composição usa diamante preenchido no lado do todo e sugere dependência forte de ciclo de vida.
- Agregação usa diamante vazio e representa todo-parte fraca.
- Diagrama de atividades modela fluxo de trabalho; não é o melhor para ordem de mensagens entre objetos.
- Componente mostra módulos e interfaces; implantação mostra nós, dispositivos, ambientes de execução e artefatos.
- Multiplicidade em diagrama de classes é restrição quantitativa; leia sempre do lado oposto da associação.

11. Reforço aprofundado por diagrama - o que a FGV espera que você diferencie

11.1 Casos de uso: requisito funcional em alto nível, não implementação

Em prova, casos de uso aparecem ligados a requisitos e análise. O objetivo não é desenhar a arquitetura interna, mas responder: **quais objetivos o ator externo consegue atingir usando o sistema?** Um caso de uso deve ter valor observável para o ator. Por isso, 'Autenticar usuário', 'Consultar processo' e 'Protocolar petição' podem ser bons candidatos; 'chamar método validar()', 'persistir entidade' ou 'renderizar componente Angular' são detalhes de projeto ou implementação.

Na prática, um time pode derivar endpoints, telas, critérios de aceite e testes a partir de casos de uso, mas a UML não exige que cada endpoint seja um caso de uso. Esse é um ponto importante para você, porque sua experiência com API pode induzir a mapear mecanicamente CRUD para caso de uso. Em prova, olhe antes para o objetivo do ator.

Sinal no enunciado	Resposta esperada	Cuidado
funcionalidades do sistema sob o ponto de vista do usuário	casos de uso	Não escolha atividade só porque há verbo de ação.
ator externo, sistema externo, papel que interage	ator de caso de uso	Ator não é classe; ator não fica dentro da fronteira.
passo obrigatório sempre reutilizado	include	Não confundir com extend.
comportamento opcional, condicional ou variante	extend	Não confundir com generalização.

11.2 Classes: estrutura estática, multiplicidade e ciclo de vida

O diagrama de classes é a ponte natural entre **análise orientada a objetos e projeto**. A FGV costuma explorar a semântica das relações: associação, agregação, composição, generalização, realização e dependência. Uma boa resposta geralmente depende de ler o tipo de linha, o lado do diamante, a multiplicidade e a direção da seta. A banca também gosta de classe associativa quando a relação possui atributos próprios.

Na sua realidade Java/Quarkus, é tentador pensar 'se eu colocaria uma Foreign Key no banco, então é composição'. Cuidado: UML não é DER - Diagrama Entidade-Relacionamento. A composição indica relação todo-parte forte e dependência de ciclo de vida no modelo; agregação indica todo-parte fraca; associação simples indica vínculo estrutural sem semântica forte de posse.

Pergunta de prova	Raciocínio
A parte pode existir sem o todo?	Se não, pense em composição; se sim, agregação ou associação.
A relação possui atributos próprios?	Pense em classe associativa.
Um elemento é tipo especializado de outro?	Pense em generalização/herança.
Uma classe implementa contrato/interface?	Pense em realização.
Um elemento apenas usa outro temporariamente?	Pense em dependência.

11.3 Sequencia: leitura temporal de cima para baixo

Diagrama de sequencia e extremamente natural para quem trabalha com APIs, mas a prova não cobra framework: cobra a semantica. A lifeline representa participante; a mensagem representa comunicacao; a barra de ativacao representa execucao; a ordem vertical representa tempo. Em uma API REST, a mensagem pode representar chamada HTTP; em mensageria, pode representar publicacao de evento; em um objeto Java, chamada de metodo. O ponto comum e a interacao ordenada.

A FGV pode tentar trocar sequencia por comunicacao. Ambos sao diagramas de interacao, mas sequencia enfatiza ordem temporal; comunicacao enfatiza enlaces entre objetos e mensagens numeradas. Se o enunciado insistir em 'ao longo do tempo', 'ordem temporal', 'linha de vida' ou 'mensagens em sequencia', a resposta e sequencia.

Fragmento	Leitura em linguagem de código	Exemplo
alt	if/else	[documento valido] autuar; [invalido] devolver para correcao
opt	if sem else	[usuario quer e-mail] enviar notificacao
loop	for/while	para cada documento enviado, validar hash
par	execucao concorrente	notificar partes e indexar documento em paralelo

11.4 Atividades: fluxo de processo, decisao e paralelismo

Diagrama de atividades e o mais proximo de um workflow. Ele pode modelar um processo de negocio do TJSC, um fluxo de tela ou uma logica de alto nivel. A notacao com losango e barras grossas costuma aparecer em questoes porque permite testar decisao/merge e fork/join. A regra: decision escolhe um caminho; merge reúne caminhos alternativos; fork divide em paralelismo; join sincroniza paralelismo.

Cuidado com a palavra 'fluxo'. Diagrama de sequencia tambem tem fluxo, mas de mensagens no tempo. Diagrama de atividades tem fluxo de controle/objetos entre acoes. Em prova, se aparecem tarefas como receber, conferir, decidir, autuar, notificar, arquivar, e o foco e processo de trabalho, escolha atividade.

Elemento	Entra no diagrama de atividades?	Observacao
Ator externo	Pode aparecer como responsavel/particao, mas não e foco principal	Foco continua no fluxo.
Classe com atributos	Não	Isso e diagrama de classes.
Decisao com guardas	Sim	Losango com [condicao].
Fork/join	Sim	Barras para paralelismo.
Mensagem HTTP ordenada	Não como foco	Melhor em sequencia.

11.5 Componentes: contratos e modularidade

Diagrama de componentes ajuda a enxergar a arquitetura logica em alto nivel. O que a FGV pode cobrar: componente, interface provida, interface requerida, porta e dependencia. Pense em blocos substituiveis que colaboram por contratos. Em sistemas corporativos, pode representar frontend, API, modulo de autenticao, modulo de documentos, integracao com servico externo e repositório.

A diferenca para classes e o nivel de abstracao: classe e menor e detalha atributos/operacoes; componente e mais arquitetural e representa unidade modular com interfaces. A diferenca para

implantacao e o ponto de vista: componente e logico; implantacao e fisico/runtime.

11.6 Implantacao: fisico, runtime e artefatos

Diagrama de implantacao e onde aparecem device, execution environment, artifact e communication path. Em uma arquitetura moderna, a FGV pode usar termos como servidor, container, pod, JVM, browser, banco, rede, HTTPS, JDBC e arquivo JAR. O importante nao e conhecer Kubernetes profundamente, mas perceber que o foco e a distribuicao dos artefatos em ambientes de execucao.

Analogia direta com seu trabalho: 'api.jar rodando em uma JVM dentro de um container em um cluster' e implantacao. 'API de Processo requer interface de Notificacao' e componente. 'Processo tem muitos Documentos' e classe. 'Usuario chama API e recebe DTO' e sequencia.

12. Quadro final de contrastes - decisao em 10 segundos

Se o enunciado disser...	Nao caia em...	Marque mentalmente
usuario, ator, funcionalidade, fronteira	classe ou atividade	caso de uso
atributo, operacao, multiplicidade, heranca	DER ou implantacao	classes
mensagem, lifeline, retorno, ordem temporal	atividade	sequencia
workflow, tarefa, decisao, fork/join	sequencia	atividades
modulo, interface provida/requerida, porta	implantacao	componentes
servidor, JVM, pod, artifact, device	componente	implantacao

Esse quadro e o seu filtro de chute consciente. Quando a questao estiver longa e voce estiver cansado, circule a palavra-chave do enunciado e compare com a ultima coluna. Em UML, muitas questoes sao vencidas por classificacao correta do objetivo do diagrama.

13. Lista de questoes autorais - sem gabarito

Resolva como se fosse prova. Nao consulte o gabarito antes de marcar as 20 respostas. As alternativas foram distribuıdas de forma equilibrada entre A, B, C, D e E.

1. No projeto de um sistema de atendimento digital do Poder Judiciario, a equipe deseja documentar, em uma visao inicial de requisitos, quais **funcionalidades** o sistema disponibiliza para partes, advogados, servidores e sistemas externos. O gerente do projeto pediu um diagrama que evidencie os agentes externos, a fronteira do sistema e os objetivos funcionais percebidos por esses agentes, sem detalhar classes internas, tabelas ou a sequencia temporal de chamadas entre servicos. Nesse contexto, o diagrama UML mais adequado e o de:

- (A) atividades, por representar o fluxo interno de cada operacao do sistema.
- (B) sequencia, por representar a ordem temporal das chamadas entre usuario e banco de dados.
- (C) casos de uso, por representar funcionalidades do sistema sob o ponto de vista de atores externos.
- (D) componentes, por representar os modulos fisicos implantados em servidores.
- (E) implantacao, por representar a alocao dos artefatos nos ambientes de execucao.

2. Considere que um modelo de classes de um sistema de processos digitais contenha as seguintes relações: (i) Processo e composto por Documento, de modo que documentos internos do processo não existem independentemente do processo no modelo; (ii) Vara agrega Servidor, pois servidores podem ser realocados e continuam existindo independentemente da vara; (iii) RecursoEspecial é uma especialização de Recurso. As três relações descritas correspondem, respectivamente, a:

- (A) composição, agregação e generalização.
- (B) agregação, composição e realização.
- (C) dependência, agregação e associação.
- (D) generalização, composição e dependência.
- (E) associação, realização e composição.

3. Uma equipe modelou a chamada feita por uma aplicação Angular para uma API em Quarkus. O desenho apresenta quatro participantes no topo - TelaConsulta, ProcessoResource, ProcessoService e ProcessoRepository -, linhas de vida verticais, barras de ativação e mensagens como consultarProcesso(id), buscarPorId(id) e retornarDTO(). A equipe quer demonstrar a ordem temporal das interações durante uma consulta. A afirmativa correta sobre esse modelo é:

- (A) trata-se de diagrama de classes, pois nomes como Resource, Service e Repository representam classes do sistema.
- (B) trata-se de diagrama de implantação, pois a chamada HTTP ocorre em ambiente distribuído.
- (C) trata-se de diagrama de atividades, pois toda chamada a API representa um fluxo de trabalho.
- (D) trata-se de diagrama de sequência, pois evidencia participantes, linhas de vida, mensagens e ordem temporal.
- (E) trata-se de diagrama de componentes, pois a API e o repositório são componentes obrigatórios da arquitetura.

4. Em um sistema de protocolo, o fluxo de cadastramento de petição foi modelado com um nó inicial, a atividade 'receber documentos', um losango que verifica se a documentação está completa, caminhos alternativos com guardas [completa] e [incompleta], uma etapa de atualização e um nó final. O diagrama também usa uma barra grossa para dividir a notificação das partes e a indexação documental em atividades paralelas. Esse conjunto de elementos caracteriza principalmente um diagrama de:

- (A) casos de uso, pois representa atores e funcionalidades externas ao sistema.
- (B) atividades, pois representa fluxo de controle, decisões e paralelismo de tarefas.
- (C) classes, pois representa atributos e operações das entidades do protocolo.
- (D) componentes, pois representa módulos de software e interfaces fornecidas.
- (E) implantação, pois mostra a distribuição dos artefatos em servidores.

5. Durante uma revisão de arquitetura, foi solicitado um diagrama UML para mostrar que o artefato api-processo.jar roda em um pod Kubernetes, que o frontend Angular é servido por um servidor web, que o banco de dados executa em outro nó e que as comunicações entre esses nós ocorrem por HTTPS e JDBC. O objetivo é evidenciar a distribuição física e os ambientes de execução. O diagrama mais adequado é o de:

- (A) casos de uso.
- (B) classes.
- (C) sequência.

- (D) atividades.
- (E) implantacao.

6. Ao revisar um diagrama de casos de uso, uma analista observou que o ator 'Sistema de Pagamentos' havia sido desenhado fora da fronteira do sistema de processos digitais e associado ao caso de uso 'Registrar custas'. Outro revisor alegou que isso estaria necessariamente errado, pois atores deveriam ser apenas pessoas. Considerando UML e a cobranca usual de prova, assinale a afirmativa correta.

- (A) O revisor esta errado, pois ator representa papel externo que interage com o sistema, podendo ser pessoa, organizacao ou outro sistema.
- (B) O revisor esta correto, pois atores UML representam exclusivamente usuarios humanos autenticados.
- (C) O ator deveria ser desenhado dentro da fronteira do sistema, pois participa da funcionalidade modelada.
- (D) A associacao entre ator e caso de uso deve ser substituida por composicao, pois ha dependencia operacional.
- (E) O caso de uso deve ser convertido em classe, pois sistemas externos so interagem com classes.

7. Em um diagrama de classes, a associacao entre Processo e Parte apresenta a multiplicidade 1 no lado de Processo e 1..* no lado de Parte. Considerando a leitura da multiplicidade em UML, e correto afirmar que:

- (A) cada parte pertence obrigatoriamente a exatamente um processo e cada processo pertence a exatamente uma parte.
- (B) um processo pode existir sem parte, desde que a parte seja cadastrada posteriormente.
- (C) cada processo esta associado a uma ou mais partes, e cada parte da associacao considerada esta vinculada a um processo.
- (D) a relacao e necessariamente de composicao, pois toda multiplicidade 1..* implica ciclo de vida dependente.
- (E) a relacao e necessariamente de generalizacao, pois Parte especializa Processo.

8. Em um diagrama de sequencia de autenticacao, a equipe usou um fragmento combinado com a palavra-chave alt para separar os caminhos [credenciais validas] e [credenciais invalidas]. Em outro trecho, usou opt para envio de notificacao quando o usuario habilitou alerta por e-mail, e loop para validar varios documentos enviados em lote. A interpretacao correta e:

- (A) alt representa repeticao, opt representa paralelismo e loop representa alternativa entre caminhos.
- (B) alt representa modularizacao de componentes, opt representa interface requerida e loop representa artefato implantado.
- (C) alt e usado apenas em diagrama de atividades, enquanto opt e loop sao exclusivos de diagrama de classes.
- (D) alt representa alternativas condicionais, opt comportamento opcional e loop repeticao.
- (E) alt, opt e loop sao estereotipos de caso de uso, sem relacao com sequencia.

9. Um analista precisa demonstrar o fluxo de tarefas de uma secretaria: receber peticao, conferir documentos, decidir se ha pendencia, autuar processo, distribuir para vara competente e comunicar as partes. O foco nao e identificar classes internas nem mensagens temporais entre objetos, mas sim representar o processo de trabalho com decisoes e possiveis caminhos. Nesse cenario, o diagrama UML mais apropriado e o de:

- (A) classes.
- (B) atividades.
- (C) componentes.
- (D) implantacao.
- (E) objetos.

10. Em uma visao logica de uma solucao corporativa, a equipe deseja mostrar que o componente 'Modulo de Assinatura' fornece a interface IAssinaturaDigital, que o componente 'Servico de Processo' requer essa interface e que ha dependencia entre os modulos. Nao se pretende mostrar servidores, pods, JVMs ou arquivos implantados. O diagrama UML mais adequado e o de:

- (A) implantacao, pois toda interface e implantada fisicamente em algum servidor.
- (B) atividades, pois interfaces representam atividades executadas pelo sistema.
- (C) sequencia, pois componentes sempre se comunicam por mensagens temporais.
- (D) casos de uso, pois a interface representa uma funcionalidade do usuario.
- (E) componentes, pois modela modulos, interfaces fornecidas/requeridas e dependencias.

11. Uma equipe desenhou um no <> ServidorAplicacao, dentro dele um <> JVM, e dentro deste o artefato processo-api.jar. Tambem desenhou um no BancoDados e um caminho de comunicacao entre ServidorAplicacao e BancoDados. A leitura correta desse diagrama e:

- (A) trata-se de diagrama de implantacao, pois mostra nos, ambiente de execucao, artefato implantado e comunicacao.
- (B) trata-se de diagrama de classes, pois JARs sao classes compiladas em Java.
- (C) trata-se de diagrama de casos de uso, pois representa funcionalidades disponiveis ao usuario.
- (D) trata-se de diagrama de atividades, pois a JVM executa atividades em tempo de execucao.
- (E) trata-se de diagrama de sequencia, pois ha comunicacao entre servidor e banco.

12. Em um diagrama de casos de uso, o caso de uso 'Protocolar peticao' sempre executa 'Validar documentos', pois a validacao e passo obrigatorio e reutilizado por outros casos de uso. Em outro ponto, 'Solicitar prioridade' ocorre apenas em certas condicoes, quando o protocolo envolve pessoa idosa ou outra hipotese legal. As relacoes mais adequadas, respectivamente, sao:

- (A) extend e include.
- (B) include e extend.
- (C) generalizacao e composicao.
- (D) agregacao e realizacao.
- (E) dependencia e associacao.

13. Considere uma classe Documento em UML, com os seguintes elementos: - hash: String, + validar(): boolean e # calcularResumo(): String. De acordo com a notacao de visibilidade de UML, esses membros indicam, respectivamente:

- (A) publico, privado e de pacote.
- (B) protegido, publico e privado.
- (C) privado, publico e protegido.
- (D) de pacote, privado e publico.
- (E) abstrato, estatico e final.

14. Em um sistema de alocação de servidores, as classes Servidor e Unidade estão associadas, e a própria associação possui atributos como dataInicio, dataFim e gratificação. O projetista deseja representar esses atributos da relação, e não de Servidor isoladamente nem de Unidade isoladamente. Em UML, a solução adequada é usar:

- (A) uma agregação compartilhada, pois toda associação com atributo é uma agregação.
- (B) uma composição, pois a relação possui ciclo de vida dependente dos objetos associados.
- (C) uma dependência tracejada, pois os atributos da relação são temporários.
- (D) uma classe associativa ligada à associação entre Servidor e Unidade.
- (E) um diagrama de implantação, pois alocação representa distribuição física.

15. Durante a modelagem de uma operação de assinatura digital, o analista quer demonstrar que o participante Usuário envia uma solicitação para Frontend, que Frontend chama API, que API chama ServiçoAssinatura, e que cada chamada possui resposta em ordem. O foco está na troca de mensagens e na ordem temporal da interação. O elemento central desse tipo de diagrama é:

- (A) multiplicidade entre classes.
- (B) fronteira do sistema e atores externos.
- (C) nós físicos e artefatos implantados.
- (D) interfaces providas e requeridas por componentes.
- (E) linhas de vida dos participantes e mensagens ordenadas no tempo.

16. Em um diagrama de atividades do fluxo de publicação de atos, as ações foram separadas em raias: Gabinete, Secretaria e Sistema. Cada raia contém as atividades de sua responsabilidade, facilitando visualizar quem executa cada etapa. Em UML, essas raias são chamadas de:

- (A) partições de atividade, também conhecidas como swimlanes.
- (B) lifelines, pois representam a existência temporal dos participantes.
- (C) interfaces requeridas, pois cada setor consome serviços de outro.
- (D) nós de implantação, pois cada setor representa um ambiente físico.
- (E) classes associativas, pois ligam atividades a responsáveis.

17. Uma equipe confundiu dois modelos: no primeiro, desenhou MóduloProcesso, MóduloDocumento e MóduloNotificação com interfaces entre si; no segundo, desenhou ServidorAplicação, Container, JVM, api.jar e BancoDados. A distinção correta entre esses modelos é:

- (A) ambos são diagramas de classes, pois todos representam partes do sistema.
- (B) o primeiro tende a ser de componentes; o segundo tende a ser de implantação.
- (C) o primeiro tende a ser de implantação; o segundo tende a ser de casos de uso.
- (D) ambos são diagramas de atividades, pois representam execução de software.
- (E) o primeiro é de sequência; o segundo é de comunicação.

18. Em uma revisão de requisitos, um servidor pergunta se o diagrama de casos de uso deve conter detalhes como nomes de tabelas, DTOs, classes Java e endpoints REST. O analista responde corretamente que esse diagrama deve manter foco em funcionalidades percebidas pelos atores. Com base nessa orientação, assinale a alternativa correta.

- (A) Casos de uso são adequados para especificar todos os atributos persistidos no banco relacional.

- (B) Casos de uso substituem completamente regras de negocio, prototipos e documentacao textual.
- (C) Casos de uso ajudam a delimitar funcionalidades e interacoes ator-sistema, mas nao detalham a implementacao interna.
- (D) Casos de uso devem representar apenas APIs REST e nunca usuarios humanos.
- (E) Casos de uso representam exclusivamente componentes implantaveis de uma arquitetura.

19. A respeito dos diagramas UML, analise as afirmativas a seguir. I. Diagrama de classes fornece uma visao estatica ou estrutural do sistema. II. Diagrama de sequencia mostra interacoes entre participantes ao longo do tempo. III. Diagrama de implantacao e indicado para mostrar a distribuicao de artefatos em nos e ambientes de execucao. Esta correto o que se afirma em:

- (A) I, apenas.
- (B) II, apenas.
- (C) I e III, apenas.
- (D) I, II e III.
- (E) II e III, apenas.

20. Um gerente afirmou que a UML define obrigatoriamente que o projeto seja executado por RUP, que substitui a necessidade de elicitar requisitos e que seus diagramas, por si so, geram codigo executavel correto em Java. Considerando o papel da UML na engenharia de software, assinale a afirmativa correta.

- (A) A afirmacao esta correta, pois UML e uma metodologia completa de desenvolvimento de software.
- (B) A afirmacao esta correta apenas para sistemas orientados a objetos escritos em Java.
- (C) A afirmacao esta correta apenas quando utilizada com ferramentas CASE homologadas.
- (D) A afirmacao esta correta porque diagramas de classe eliminam a necessidade de testes.
- (E) A afirmacao esta incorreta, pois UML e linguagem de modelagem e documentacao, nao metodologia obrigatoria nem garantia automatica de implementacao correta.

14. Gabarito resumido

1-C	2-A	3-D	4-B	5-E
6-A	7-C	8-D	9-B	10-E
11-A	12-B	13-C	14-D	15-E
16-A	17-B	18-C	19-D	20-E

15. Comentarios completos

Questao 1 - Gabarito: C

Por que a correta esta correta: Caso de uso modela funcionalidades sob ponto de vista de atores externos e delimita a fronteira do sistema.

Por que as demais estão erradas:

- A erra porque atividade modela fluxo de trabalho, não mapa inicial de funcionalidades por ator.
- B erra porque sequência e para ordem temporal de mensagens.
- D erra porque componentes modela módulos e interfaces.

- E erra porque implantacao modela distribuicao fisica/runtime.

Erro provavel do candidato: confusao entre diagramas.

Questao 2 - Gabarito: A

Por que a correta esta correta: Documento depende fortemente de Processo no modelo: composicao. Vara-Servidor e todo-parte fraca: agregacao. RecursoEspecial e tipo de Recurso: generalizacao.

Por que as demais estão erradas:

- B inverte composicao/agregacao e inclui realizacao indevida.
- C confunde relacoes estruturais com dependencia/associacao generica.
- D coloca generalizacao onde ha todo-parte.
- E mistura associacao, realizacao e composicao sem correspondencia.

Erro provavel do candidato: confusao entre agregacao, composicao e heranca.

Questao 3 - Gabarito: D

Por que a correta esta correta: O enunciado descreve lifelines, mensagens e ordem temporal, elementos tipicos de diagrama de sequencia.

Por que as demais estão erradas:

- A erra porque classes podem aparecer como participantes, mas o foco nao e estrutura estatica.
- B erra porque distribuicao fisica nao foi modelada.
- C erra porque nao ha workflow com decisoes/fork/join.
- E erra porque a existencia de camadas nao transforma o desenho em componente.

Erro provavel do candidato: leitura apressada.

Questao 4 - Gabarito: B

Por que a correta esta correta: No inicial, acoes, decisao com guardas, caminhos alternativos e paralelismo caracterizam diagrama de atividades.

Por que as demais estão erradas:

- A erra porque casos de uso nao detalham fluxo interno.
- C erra porque nao ha atributos/operacoes/multiplicidades.
- D erra porque componentes envolve modulos e interfaces.
- E erra porque implantacao envolve nos e artefatos.

Erro provavel do candidato: conceito.

Questao 5 - Gabarito: E

Por que a correta esta correta: Artefatos, pods, servidores, banco, HTTPS e JDBC como comunicacao entre nos indicam diagrama de implantacao.

Por que as demais estão erradas:

- A erra por falar de funcionalidades de atores.
- B erra por falar de classes e relacionamentos estaticos.
- C erra por mensagens temporais.
- D erra por workflow de atividades.

Erro provavel do candidato: pegadinha componente vs implantacao.

Questao 6 - Gabarito: A

Por que a correta esta correta: Ator e papel externo; pode ser humano, organizacao ou outro sistema. Deve ficar fora da fronteira do sistema.

Por que as demais estão erradas:

- B erra ao limitar ator a humano.
- C erra porque ator fica fora da fronteira.
- D erra porque composicao e relacao de classes todo-parte.
- E erra porque caso de uso nao precisa virar classe.

Erro provavel do candidato: lacuna de teoria.

Questao 7 - Gabarito: C

Por que a correta esta correta: A multiplicidade no lado de Parte indica quantas partes um processo pode ter; 1..* significa uma ou mais.

Por que as demais estão erradas:

- A erra ao inverter/superrestringir a relacao.
- B erra porque 1..* exige pelo menos uma parte.
- D erra porque multiplicidade nao define composicao.
- E erra porque nao ha relacao de heranca.

Erro provavel do candidato: leitura de multiplicidade.

Questao 8 - Gabarito: D

Por que a correta esta correta: alt = alternativas condicionais; opt = comportamento opcional; loop = repeticao.

Por que as demais estão erradas:

- A troca os significados.
- B mistura conceitos de componentes.
- C erra porque fragmentos combinados pertencem a interacoes/diagrama de sequencia.
- E erra porque nao sao estereotipos de caso de uso.

Erro provavel do candidato: memorizacao.

Questao 9 - Gabarito: B

Por que a correta esta correta: Fluxo de tarefas com decisoes e caminhos caracteriza diagrama de atividades.

Por que as demais estão erradas:

- A erra: classes sao estrutura estatica.
- C erra: componentes sao modulos/interfaces.
- D erra: implantacao e fisica/runtime.
- E erra: objetos mostram instancias.

Erro provavel do candidato: conceito.

Questao 10 - Gabarito: E

Por que a correta esta correta: Componentes modelam modulos e interfaces fornecidas/requeridas, sem necessidade de mostrar onde rodam.

Por que as demais estão erradas:

- A erra porque implantação exigiria nos/artefatos/ambientes.
- B erra por confundir interface com atividade.
- C erra porque sequência exigiria ordem temporal de mensagens.
- D erra porque caso de uso e funcionalidade de ator.

Erro provável do candidato: pegadinha componente vs implantação.

Questão 11 - Gabarito: A

Por que a correta está correta: No device, execution environment, artefato JAR e caminho de comunicação são elementos de implantação.

Por que as demais estão erradas:

- B erra: JAR pode conter classes, mas o diagrama é de implantação.
- C erra por não haver atores/casos de uso.
- D erra por não haver workflow.
- E erra por não haver lifelines/mensagens ordenadas.

Erro provável do candidato: confusão entre comunicação e sequência.

Questão 12 - Gabarito: B

Por que a correta está correta: Validar documentos e comportamento obrigatório/reutilizável: include. Solicitar prioridade e condicional/opcional: extend.

Por que as demais estão erradas:

- A inverte include e extend.
- C usa generalização/composição, conceitos inadequados aqui.
- D agrega relações de classe/componente.
- E usa dependência/associação genéricas sem semântica correta.

Erro provável do candidato: pegadinha FGV.

Questão 13 - Gabarito: C

Por que a correta está correta: Em UML, '-' indica privado, '+' público e '#' protegido.

Por que as demais estão erradas:

- A inverte os sinais.
- B coloca protegido no primeiro item indevidamente.
- D usa pacote/público em posições erradas.
- E mistura modificadores de linguagem que não correspondem a esses símbolos.

Erro provável do candidato: memorização.

Questão 14 - Gabarito: D

Por que a correta está correta: Classe associativa é usada quando a associação em si possui atributos.

Por que as demais estão erradas:

- A erra porque atributo na associação não implica agregação.
- B erra porque não há ciclo de vida forte indicado.
- C erra porque dependência e uso fraco/temporário.

- E erra porque alocação lógica não é implantação física.

Erro provável do candidato: conceito.

Questão 15 - Gabarito: E

Por que a correta está correta: Diagrama de sequência tem como foco lifelines e mensagens em ordem temporal.

Por que as demais estão erradas:

- A é próprio de classes.
- B é próprio de casos de uso.
- C é próprio de implantação.
- D é próprio de componentes.

Erro provável do candidato: associação por palavra-chave.

Questão 16 - Gabarito: A

Por que a correta está correta: Raias em diagrama de atividades são partições de atividade ou swimlanes.

Por que as demais estão erradas:

- B erra: lifelines são de sequência.
- C erra: interfaces são de componentes.
- D erra: nós são de implantação.
- E erra: classe associativa é de classes.

Erro provável do candidato: termo técnico.

Questão 17 - Gabarito: B

Por que a correta está correta: Módulo + interfaces indica componentes; servidor/container/JVM/JAR/banco indica implantação.

Por que as demais estão erradas:

- A erra ao reduzir tudo a classes.
- C inverte os conceitos.
- D erra: atividade e fluxo de trabalho.
- E erra: sequência/comunicação mostram interações, não essa distribuição.

Erro provável do candidato: pegadinha componente vs implantação.

Questão 18 - Gabarito: C

Por que a correta está correta: Caso de uso delimita funcionalidades e interações ator-sistema, mas não detalha implementação interna.

Por que as demais estão erradas:

- A erra: atributos persistidos pertencem a modelagem de dados/classes.
- B erra: casos de uso não substituem toda documentação.
- D erra: atores podem ser humanos e outros sistemas; casos de uso não são apenas APIs REST.
- E erra: componentes implantáveis não são casos de uso.

Erro provável do candidato: prática real vs prova.

Questao 19 - Gabarito: D

Por que a correta esta correta: As tres afirmativas estao corretas: classes = estatica; sequencia = interacao temporal; implantacao = artefatos em nos/ambientes.

Por que as demais estao erradas:

- A ignora II e III corretas.
- B ignora I e III corretas.
- C ignora II correta.
- E ignora I correta.

Erro provavel do candidato: questao de itens.

Questao 20 - Gabarito: E

Por que a correta esta correta: UML e linguagem de modelagem/documentacao; nao obriga RUP, nao substitui requisitos e nao garante codigo correto.

Por que as demais estao erradas:

- A erra: UML nao e metodologia completa.
- B erra: UML nao se limita a Java.
- C erra: ferramenta CASE pode apoiar, mas nao torna a afirmacao verdadeira.
- D erra: diagramas nao eliminam testes.

Erro provavel do candidato: conceito basico.

16. Flashcards finais

Frete	Verso
UML e uma metodologia?	Nao. UML e linguagem de modelagem para visualizar, especificar, construir e documentar artefatos.
Qual diagrama mostra funcionalidades sob ponto de vista do usuario?	Diagrama de casos de uso.
Ator precisa ser humano?	Nao. Pode ser pessoa, organizacao, dispositivo ou outro sistema externo.
<<include>> significa o que?	Comportamento obrigatorio/reutilizado incluido pelo caso base.
<<extend>> significa o que?	Comportamento opcional/condicional que estende um caso base.
Qual diagrama mostra classes, atributos e operacoes?	Diagrama de classes.
Diamante preenchido significa o que?	Composicao: todo-parte forte, diamante no lado do todo.
Diamante vazio significa o que?	Agregacao: todo-parte fraca.
Qual diagrama mostra mensagens ao longo do tempo?	Diagrama de sequencia.
alt, opt e loop aparecem onde?	Fragmentos combinados em diagramas de sequencia/interacao.
Qual diagrama modela workflow com decisoes e paralelismo?	Diagrama de atividades.
Fork e join indicam o que?	Divisao e sincronizacao de fluxos paralelos em atividades.
Componente ou implantacao: interfaces providas/requeridas?	Componente.
Componente ou implantacao: servidor, JVM, JAR e pod?	Implantacao.
Classe associativa serve para que?	Representar atributos da associacao entre classes.

17. Checklist do que memorizar para acertar questões

- Casos de uso: ator externo, fronteira do sistema, funcionalidade, include, extend e generalizacao.
- Classes: visibilidade (+, -, #, ~), atributos, operacoes, multiplicidade, associacao, agregacao, composicao, generalizacao, realizacao, dependencia e classe associativa.
- Sequencia: lifeline, mensagem, ativacao, retorno, leitura de cima para baixo, alt, opt e loop.
- Atividades: no inicial/final, acao, decisao, merge, fork, join, particoes/swimlanes e fluxo de objeto.
- Componentes: modulos, interfaces providas/requeridas, portas e dependencias.
- Implantacao: node, device, execution environment, artifact e communication path.
- Pegadinha principal: componente = logico/modular; implantacao = fisico/runtime.
- Pegadinha secundaria: atividade = fluxo de trabalho; sequencia = mensagens no tempo.

18. Referencias utilizadas e criterio de atualizacao

Esta aula foi produzida com base no edital retificado TJSC 2026, em provas recentes da FGV com cobranca de UML e na especificacao oficial da Object Management Group - OMG. O edital cita UML versao 2.1; a FGV, em cadernos recentes, tambem utiliza linguagem de UML 2.5/2.5.1. Para fins de prova, foram priorizados conceitos estaveis de UML 2.x: finalidade dos diagramas, elementos e semantica de relacionamentos.

Fonte	Uso na aula
Edital TJSC 2026 - FGV - Analista de Sistemas	Confirmacao da cobranca de UML no bloco de Engenharia de Software e Desenvolvimento.
OMG - Unified Modeling Language Specification 2.5.1	Referencia normativa para finalidade geral da UML e familias/termos de diagramas.
Cadernos oficiais FGV - provas de Analista de Sistemas/TI	Observacao do estilo de enunciado: diagramas contextualizados, itens I/II/III, relacao de diagramas a caracteristicas e alternativas com termos proximos.
Experiencia pratica em sistemas Java/Quarkus, Angular/TypeScript, APIs e integracoes	Exemplos corporativos e analogias para fixacao.